

System analysis and Design,

Lecture-2

By

Sourav Das (SDS)

Outlines

- System Analyst
- Methodologies, Techniques and Tools
- SDLC
- Products of SDLC
- Complexity Analysis

System Analyst

- The **systems analyst** is a key person analyzing the business, identifying opportunities for improvement, and designing information systems to implement these ideas.
- It is important to understand and develop through practice the skills needed to successfully design and implement new information systems.

System Failure

- Many failed systems were abandoned because analysts tried to build **wonderful** systems **without understanding the organization**.
- The primarily goal is to create **value for the organization**.
- Quality is satisfaction of requirements, not 'goodness'

Methodologies, Techniques and Tools



FIGURE An organizational approach to systems analysis and design is driven by methodologies, techniques, and tools

Methodologies, Techniques and Tools

- **Methodologies** are comprehensive, multiple-step approaches to systems development.
- **Techniques** are particular processes that you, as an analyst, will follow to help ensure that your work is well thought out.
- **Tools** are typically computer programs that make it easy to use and benefit from techniques and to faithfully follow the guidelines of the overall development methodology.
- These three elements—methodologies, techniques, and tools—work together to form an organizational approach to systems analysis and design

THE SYSTEMS DEVELOPMENT LIFE CYCLE (SDLC)

Or Software Development Life Cycle

- The SDLC is generally presented as an iterative sequence of five steps:
 - Systems Planning
 - Systems Analysis
 - Systems Design
 - Systems Implementation
 - Maintenance,
- Each concluding in a deliverable, either a written document or a piece of software (or both)

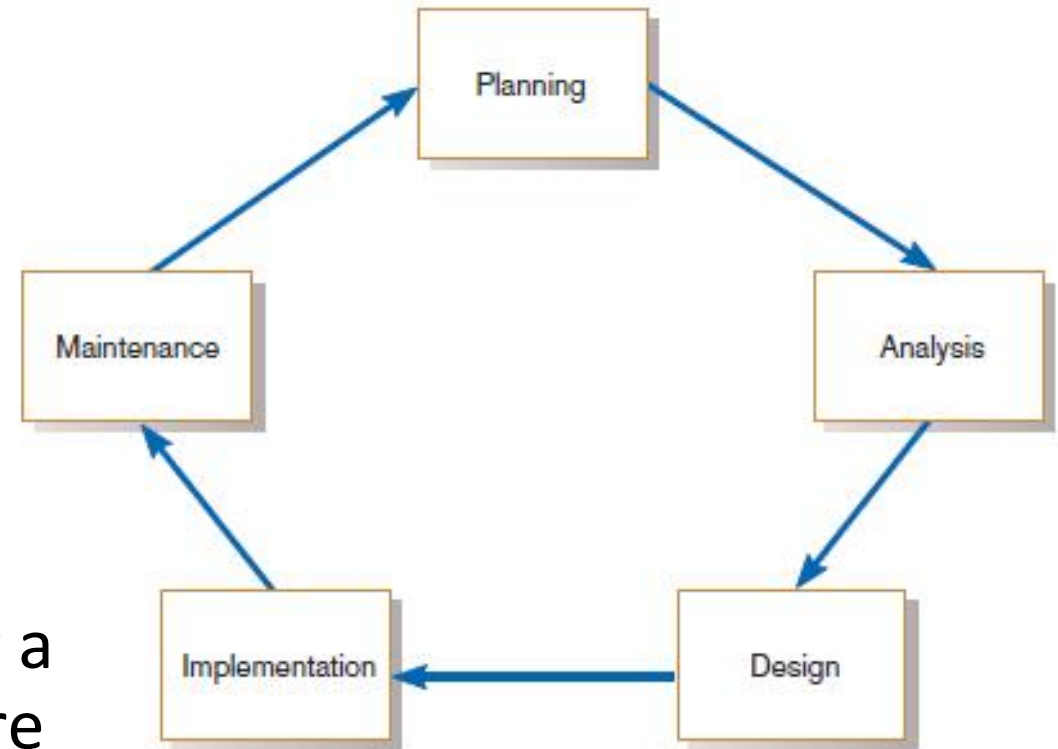


FIGURE
The systems development life cycle

THE SYSTEMS DEVELOPMENT LIFE CYCLE (SDLC)

- SDLC allows organizations to incorporate new requirements, technology and human resources to IT development

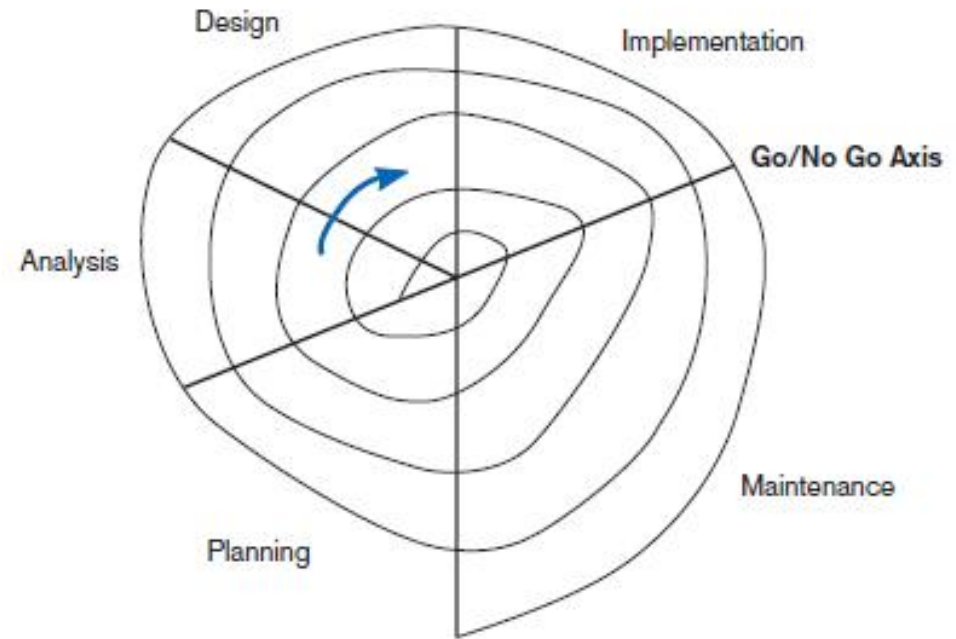


FIGURE Evolutionary model

Systems Development Life Cycle (SDLC) (Cont.)

1. **Planning** – an organization's total information system needs are identified, analyzed, prioritized, and arranged
 2. **Analysis** – system requirements are studied and structured
 3. **Design** – a description of the recommended solution is converted into logical and then physical system specifications
- ✓ **Logical design** – all functional **features** of the system chosen for development in analysis are described independently of any computer platform

Systems Development Life Cycle (SDLC)

(Cont.)

- ✓ **Physical design** – the logical specifications of the system from logical design are transformed into the technology-specific details from which all programming and system construction can be accomplished

Systems Development Life Cycle (SDLC) (Cont.)

- 4. Implementation** – the information system is **coded, tested, installed and supported** in the organization
- 5. Maintenance** – an information system is systematically repaired and improved

Systems Development Life Cycle (SDLC) (Cont.)

1. Planning – an organization's total information system needs are identified, analyzed, prioritized, and arranged

Output - Presentation

- **Investigation** of the system problem
- Presentation of reasons **why the system should or should not be developed** by the organization
- Organization's information systems departments **ability** to develop that system
- That will **solve** the problem or **exploit** the opportunity
- Determine whether the costs of developing the system outweigh the **benefits** it could provide
- The final presentation to the management or to a special management committee with the job of **deciding which projects** the organization will **undertake**.

Systems Development Life Cycle (SDLC) (Cont.)

2. Analysis – system requirements are studied and structured

Two phases

- i. **Requirements** determination/analysis.
- ii. **Structuring** them according to their interrelationships and eliminate any redundancies.

Output – Document

- Description of current system with problems and fixing opportunities
- Explanation of alternative systems and justification for chosen alternative

Systems Development Life Cycle (SDLC) (Cont.)

3. Design – a description of the recommended solution is converted into logical and then physical system specifications

Output – BRD and SRS Documents, DFD & UML Diagrams

- a) Business Requirement Document is termed as BRD. This document serves to find out practical requirements which may occur while developing any software. It elaborates on the interest of user requirements and developer requirements.
- b) Following are some features of SRS :
 - It is one of few documents created in any software development life-cycle.
 - Documents features a company can provide a workable solution.
 - It is used to find out what is expected from the system.
 - It helps in finding out project sponsors.

Systems Development Life Cycle (SDLC) (Cont.)

3. Design – a description of the recommended solution is converted into logical and then physical system specifications

Output – BRD and SRS Documents

Example : Consider software for tracking Employees spend time. This document will picturize how a system can be made which can improve efficiency of tracking an employee.

Systems Development Life Cycle (SDLC) (Cont.)

b) Software Requirement Specification (SRS) :

Software Requirement Specification is termed as SRS document. This document serves as a detailed illustration of **functional and non-functional requirements** needed that the software should fulfil.

Following are some features of SRS :

- This document bridges gap between user and the developer.
- Documents board imaginations into a structural layout.
- Used for measuring initial costs and efforts.
- Works as an agreement between communicating parties.

Systems Development Life Cycle (SDLC) (Cont.)

b) Software Requirement Specification (SRS) :

Example : Consider a software to monitor employee performance. This will require basic modules such as Login Module, Administrator Module, Employee Module, and Reporting Module. SRS document helps to manage these modules.

We will discuss DFD and UML in future

Products of (SDLC) (Cont.)

TABLE 1-1 Products of SDLC Phases

Phase	Products, Outputs, or Deliverables
Planning	Priorities for systems and projects; an architecture for data, networks, and selection hardware, and IS management are the result of associated systems Detailed steps, or work plan, for project Specification of system scope and planning and high-level system requirements or features Assignment of team members and other resources System justification or business case
Analysis	Description of current system and where problems or opportunities are with a general recommendation on how to fix, enhance, or replace current system Explanation of alternative systems and justification for chosen alternative
Design	Functional, detailed specifications of all system elements (data, processes, inputs, and outputs) Technical, detailed specifications of all system elements (programs, files, network, system software, etc.) Acquisition plan for new technology
Implementation	Code, documentation, training procedures, and support capabilities
Maintenance	New versions or releases of software with associated updates to documentation, training, and support

SDLC and System Development Methodology

- Systems development life cycle (SDLC)
The traditional methodology used to develop, maintain, and replace information systems.
- Systems development methodology

A standard process followed in an organization to conduct all the steps necessary to analyse, design, implement, and maintain information systems

WHY BUILD MODELS?

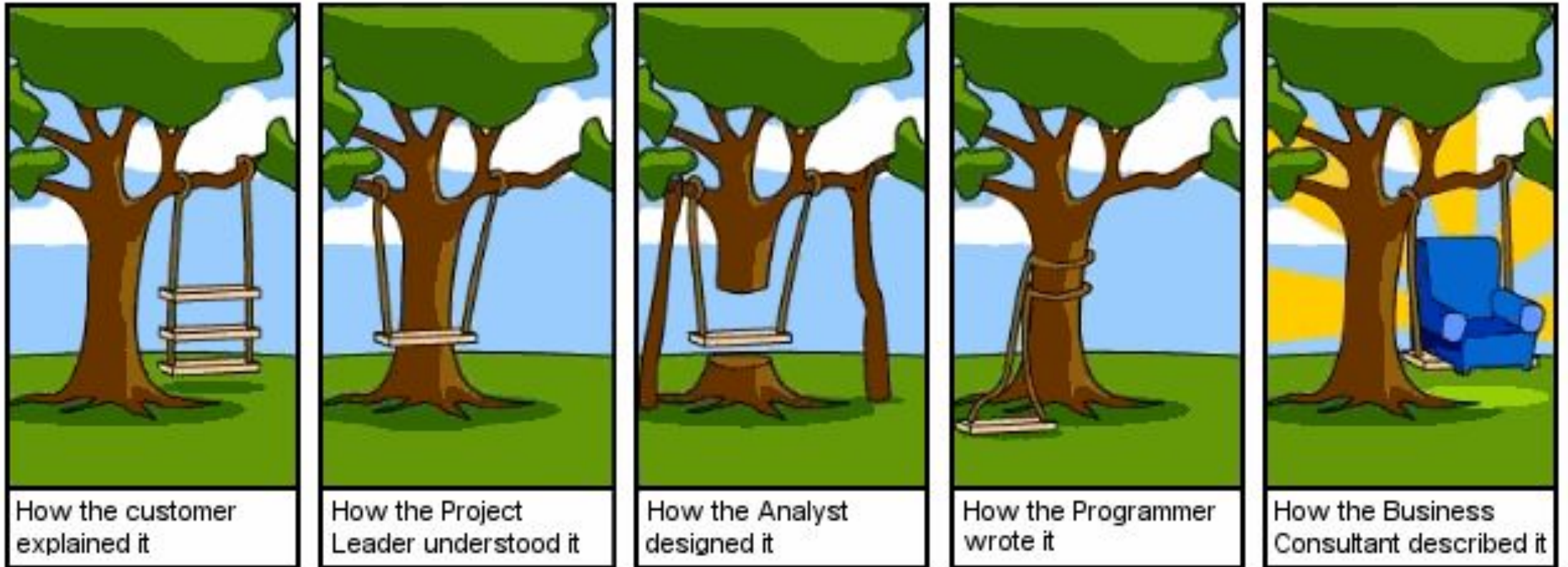


How the customer
explained it.

**What do you
think is the
problem here?**

Why?

WHY BUILD MODELS?



SYSTEMATIC APPROACH IS NEEDED TO AVOID THIS

WHY BUILD MODELS?

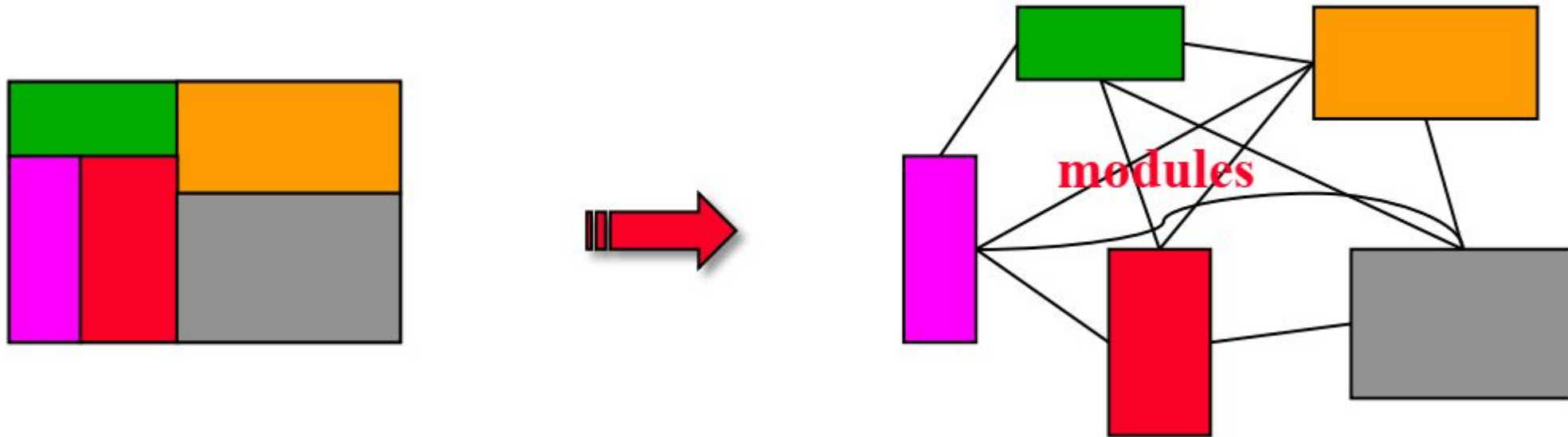
- ❑ Models succinctly describe reality (i.e., they abstract reality).
- ❑ They show essential details and filter out non-essential details.
For software development, this allows us to focus on the “big picture”, i.e. programming-in-the-large.
- ❑ Such a focus allows us to better deal with the *complexity* of software development, i.e., with human limitations in understanding complex things.
- ❑ The result is better understanding of requirements, cleaner designs, and more maintainable systems.

SOFTWARE IS COMPLEX TO DEVELOP

- Boeing 787: 14 MLOC
- F-35 Fighter Jet: 24 MLOC
- Windows 7: ~40MLOC
- Windows 10: ~50 MLOC
- Facebook: 61 MLOC
- Mac OS X: ~90 MLOC
- Luxury passenger car: 100 MLOC

MLOC= million Line of code

DEALING WITH COMPLEXITY: MODULAR & INCREMENTAL DEVELOPMENT

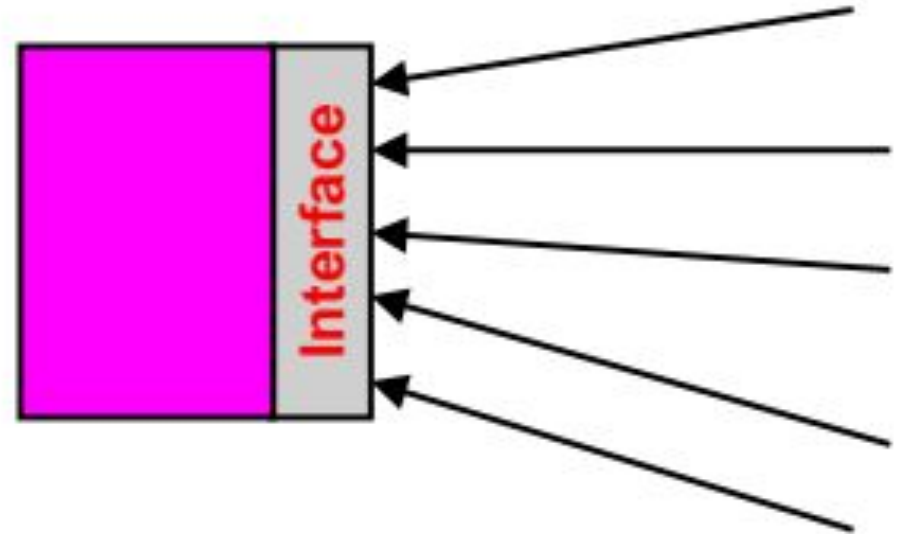


module: A part of a system that it **makes sense to consider separately**.

👉 **BUT** modules need to **interact** with each other.

DEALING WITH COMPLEXITY: MODULAR & INCREMENTAL DEVELOPMENT (cont'd)

- There is a limit to human understanding.
- **USE INFORMATION HIDING**
- Allow **modules** to interact *only* **via interfaces**.
- An interface **abstracts** and **encapsulates** a module thereby providing information hiding



DEALING WITH COMPLEXITY: MODULAR & INCREMENTAL DEVELOPMENT (cont'd)

- An **interface abstracts a module** so the **user does not have to know** *how* the module is implemented to **use it**.
- A module can be used by **understanding** *only* its **interface**.
- This reduces the **complexity** of **understanding** the system!
- An interface encapsulates a module so the user cannot use knowledge about how the module is implemented
- A module can be changed (internally) without affecting the rest of the system.
- This reduces the complexity of maintaining the system!
- This also reduces the complexity of cost and time estimates for developing the system!

DEALING WITH COMPLEXITY: MODULAR & INCREMENTAL DEVELOPMENT (cont'd)

- Modular and incremental development using interfaces allows for:
 - more productivity in team development
 - fewer bugs in system development
 - more maintainable software
 - more reusable software
- Results in more predictable software development.